

CS112: Monday 31 August, 12:15 to 1:20

DRAFT, RECONSTRUCTED FROM COURSE-MAP.YML AND THE TEACHING LOG

You arrive from Python, where running a program is one step. You wrote the file, you typed `python thing.py`, and it ran. Nothing stood between the text you wrote and the thing that happened.

Today: your source file is not the program. Something has to turn it into one, and that something is a chain of tools with three separate places to go wrong. Knowing which stage failed is most of what reading a C++ error message is.

TIME	BEAT
12:20	1. The shape of a C++ program
12:32	2. The pipeline, and where it breaks
12:50	3. Errors, live

Beat 1. The shape of a C++ program (12 min)

```
#include <iostream>
using namespace std;

int main() {
    cout << "hello" << endl;
    return 0;
}
```

Four things worth naming while typing it: `#include` pulls in a file before compilation begins; `main` is where the program starts and it returns an `int`; `cout <<` sends a value out, `cin >>` reads one in; `return 0` is how a program reports that nothing went wrong.

Talk to your neighbor. What does the compiler actually do to this file?

Beat 2. The pipeline, and where it breaks (18 min)

```
hello.cpp --preprocess--> --compile--> hello.o --link-->
hello
```

Each arrow can fail, and they fail differently.

- A **compile error** means one file does not make sense on its own: a missing semicolon, a name never declared, a type mismatch. The compiler names the file and the line.
- A **link error** means every file made sense alone and together they are missing something: a function declared and never written. There is no line number, because there is no line; the thing is simply not there.

That distinction is the whole reason `make` exists. It knows which files changed and which step each one needs next.

Beat 3. Errors, live (15 min)

Break the program on purpose and read what comes back, one at a time. Delete a semicolon. Misspell `cout`. Declare a function and never define it.

Which stage failed, and how can you tell from the message alone?

The habit worth starting today: **read the first error, not the last one.** One missing semicolon produces a cascade, and every message after the first is the compiler guessing about wreckage it caused itself.